

ОТЧЕТ
по лабораторной работе № 9
по дисциплине «Программирование на Python»

Тема: CRUD для приложения отслеживания курсов валют с SQLite базой
данных

Выполнил: студент гр. Р3121
Баранова Елизавета Павловна

Преподаватель: Жуков Николай
Николаевич

1. Цели работы

1. Реализовать **CRUD** (Create, Read, Update, Delete) для сущностей бизнес-логики приложения.
2. Освоить работу с **SQLite** в памяти (:memory:) через модуль sqlite3.
3. Понять принципы **первичных и внешних ключей** и их роль в связях между таблицами.
4. Выделить **контроллеры** для работы с БД и для рендеринга страниц в отдельные модули.
5. Использовать архитектуру **MVC** и соблюдать разделение ответственности.
6. Отображать пользователям таблицу с валютами, на которые они подписаны.
7. Реализовать полноценный **роутер**, который обрабатывает GET-запросы и выполняет сохранение/обновление данных и рендеринг страниц.
8. Научиться тестировать функционал на примере сущностей currency и user с использованием **unittest.mock**.

2. Описание моделей, их свойств и связей

Приложение моделирует систему управления пользователями и их подписками на валюты:

Author — автор приложения, свойства: name, group.

App — приложение, свойства: name, version, author (связь с объектом Author).

User — пользователь, свойства: id, name.

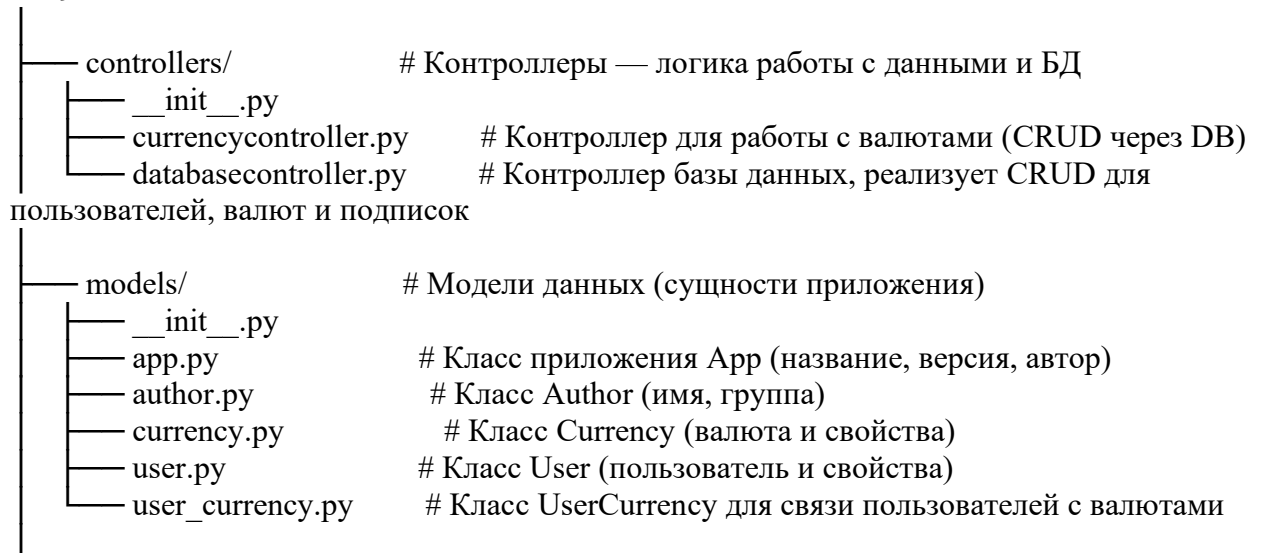
Currency — валюта, свойства: id, num_code, char_code, name, value, nominal.

UserCurrency — связь пользователя с валютой, свойства: id, user_id, currency_id.

Один пользователь может быть подписан на несколько валют

3. Структура проекта

Lab9/



— templates/	# HTML-шаблоны для рендеринга страниц
— author.html	# Страница "Об авторе"
— currencies.html	# Страница со списком валют, формами для изменения
— index.html	# Главная страница приложения
— user_subscriptions.html	# Страница подписок конкретного пользователя
— users.html	# Страница со списком пользователей
— tests/	# Тесты приложения
— test_crud.py	# Тесты CRUD операций для базы данных и контроллеров
— test.py	# Другие тесты
— utils/	# Утилиты
— currencies_api.py	# Функция get_currencies для получения курсов с API ЦБ
— __init__.py	
— myapp.py	# Основной скрипт запуска веб-сервера, маршрутизация
GET/POST	
— README.md	# Документация проекта, инструкция по запуску и описания

4. Реализация CRUD

1. Создание валюты (Create)

Метод: `currency_create(num_code, char_code, name, value=None, nominal=1)`

Добавляет новую валюту в таблицу `currency`.

Если `value` не указан, берётся текущий курс с API ЦБ.

2. Чтение всех валют (Read)

Метод: `currency_read_all()`

Возвращает список всех валют из таблицы `currency`.

3. Обновление курса валюты (Update)

Метод: `currency_update(char_code, new_value=None)`

Обновляет значение `value` валюты с заданным `char_code`.

Если `new_value=None`, берётся актуальный курс с API ЦБ.

4. Удаление валюты (Delete)

Метод: `currency_delete(currency_id)`

Удаляет валюту по `id` из таблицы `currency`.

5. Скриншоты работы приложения

Приложение для отслеживания курсов валют

Автор проекта: Liza

Группа: P3121

- [Главная](#)
- [Пользователи](#)
- [Курсы валют](#)
- [Об авторе проекта](#)

<

Список пользователей

- [Alexandr](#)
- [Vasiliy](#)

[Главная](#)

Валюты

Добавить новую валюту

CharCode: Название: Курс (оставьте пустым для автообновления с ЦБ): Номинал:

ID	CharCode	Name	Value	Nominal	Обновить с ЦБ	Удалить
1	USD	Доллар США	76,0937 Обновить	1	Обновить с ЦБ	Удалить
2	EUR	Евро	88,7028 Обновить	1	Обновить с ЦБ	Удалить
3	GBP	Фунт стерлингов	101,7601 Обновить	1	Обновить с ЦБ	Удалить

Валюты

Добавить новую валюту

CharCode: Название: Курс (оставьте пустым для автообновления с ЦБ): Номинал:

ID	CharCode	Name	Value	Nominal	Обновить с ЦБ	Удалить
1	USD	Доллар США	76,0937 Обновить	1	Обновить с ЦБ	Удалить
2	EUR	Евро	88,7028 Обновить	1	Обновить с ЦБ	Удалить
3	GBP	Фунт стерлингов	101,7601 Обновить	1	Обновить с ЦБ	Удалить
4	AUD	Австралийский доллар	50,374 Обновить	1	Обновить с ЦБ	Удалить

Результат добавления валюты

Валюты

Добавить новую валюту

CharCode: Название: Курс (оставьте пустым для автообновления с ЦБ): Номинал:

ID	CharCode	Name	Value	Nominal	Обновить с ЦБ	Удалить
1	USD	Доллар США	76,0937 Обновить	1	Обновить с ЦБ	Удалить
3	GBP	Фунт стерлингов	101,7601 Обновить	1	Обновить с ЦБ	Удалить
4	AUD	Австралийский доллар	50,374 Обновить	1	Обновить с ЦБ	Удалить

Результат удаления валюты

```
127.0.0.1 - - [06/Dec/2025 23:08:40] "GET /users HTTP/1.1" 200 -
127.0.0.1 - - [06/Dec/2025 23:08:43] "GET /currencies HTTP/1.1" 200 -
127.0.0.1 - - [06/Dec/2025 23:09:24] "POST /currencies HTTP/1.1" 303 -
127.0.0.1 - - [06/Dec/2025 23:09:24] "GET /currencies HTTP/1.1" 200 -
127.0.0.1 - - [06/Dec/2025 23:09:32] "POST /currencies HTTP/1.1" 303 -
127.0.0.1 - - [06/Dec/2025 23:09:32] "GET /currencies HTTP/1.1" 200 -
```

6. Примеры тестов

```
class TestCurrencyController(unittest.TestCase):
    def setUp(self):
        self.mock_db = MagicMock(spec=DatabaseController)
        self.controller = CurrencyController(self.mock_db)

    def test_list_currencies(self):
        """Проверка получения списка валют."""
        self.mock_db.currency_read_all.return_value = [
            {"id": 1, "char_code": "USD", "name": "Dollar", "value": 100,
"nominal": 1},
            {"id": 2, "char_code": "EUR", "name": "Euro", "value": 95,
"nominal": 1}
        ]
        result = self.controller.list_currencies()
        self.assertEqual(len(result), 2)
        self.assertEqual(result[0]['char_code'], "USD")
        self.mock_db.currency_read_all.assert_called_once()

    def test_list_empty(self):
        """Чтение списка валют, если база пустая."""
        self.mock_db.currency_read_all.return_value = []
        result = self.controller.list_currencies()
        self.assertEqual(result, [])
        self.mock_db.currency_read_all.assert_called_once()
```

Ran 9 tests in 0.013s

OK

Process finished with exit code 0

7. Выводы

- Model отвечает за работу с данными: хранение пользователей, валют и подписок в SQLite.
- View формирует HTML-страницы с помощью Jinja2, используя шаблоны.
- Controller обрабатывает действия пользователя (добавление, обновление, удаление валют).
- Такой подход упрощает поддержку и расширение приложения, так как каждая часть системы отвечает за свою зону ответственности.
- SQLite обеспечивает надежное хранение данных без необходимости настройки отдельного сервера БД, что удобно для небольших локальных приложений.
- Применение шаблонов упрощает поддержку сложных интерфейсов с динамическими таблицами, формами и навигацией.
- Использование unittest и mock облегчает тестирование контроллеров без зависимости от реальной базы данных и внешних API.